

PYTHON PROJECT REPORT

Submitted By:

Name: Pritam Satish Chore

Registration number: 26BCE10997

Course: Btech. Computer Science Engineering [CORE]

Year: 1st year

Subject: Python Programming

Project Title: STUDENT HELPER

• **Table of Content**

1. Cover page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
 - Use case Diagram
 - Workflow Diagram
 - Class/Component Diagram
 - ER Diagram
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learning and Key Takeaways
14. Future Enchantments
15. References

• Introduction

Student Helper is a menu-driven Python console application designed to provide three student-oriented utilities in one program: grade evaluation, attendance tracking, and FFCS slot calculation. The supplied source implements these utilities using basic Python constructs such as variables, lists, loops, conditional statements, formatted input/output, and arithmetic operations.

The program starts by displaying a **STUDENT HELPER** menu with three choices. After completing a selected operation, the user is asked whether the program should continue. Entering N/n ends the loop and prints a thank-you message.

The report describes the project as implemented in the supplied source file; it does not assume a database, GUI, web server, or external package because none of those components appear in the source.

• Problem Statement

Students often need quick calculations related to academic performance and course registration. In the supplied project, these needs are addressed through a single console menu rather than separate manual calculations.

- **Grade evaluation:** compare each student's mark with the class average and assign a grade from S to F using the implemented ranges.
- **Attendance tracking:** calculate current attendance percentage and estimate classes that need to be attended or can be missed under the implemented 75% criterion.
- **FFCS assistance:** map a student's entered grade and total attendance to an FFCS slot according to the conditional rules encoded in the program.

The central problem addressed by the project is therefore the consolidation of these repeated student calculations into one interactive Python program.

• Functional Requirements

Requirement	Implemented Behavior
Display main menu	Shows Grade Evaluation, Attendance Tracking, and FFCS options.
Accept menu choice	Reads the user's preference as an integer.
Evaluate grades	Collects course name, mark, and class average for each registered course.
Assign grade	Uses conditional ranges to append S/A/B/C/D/F.
Generate report card	Prints course, mark, class average, and calculated grade.
Calculate attendance	Computes current attendance as $\text{attended/scheduled} \times 100$.
Check 75% criterion	Calculates attendance requirements using scheduled and upcoming classes.
Calculate FFCS slot	Uses grade and attendance combinations to print a slot number.
Repeat or exit	Asks for Y/N and continues unless N/n is entered.

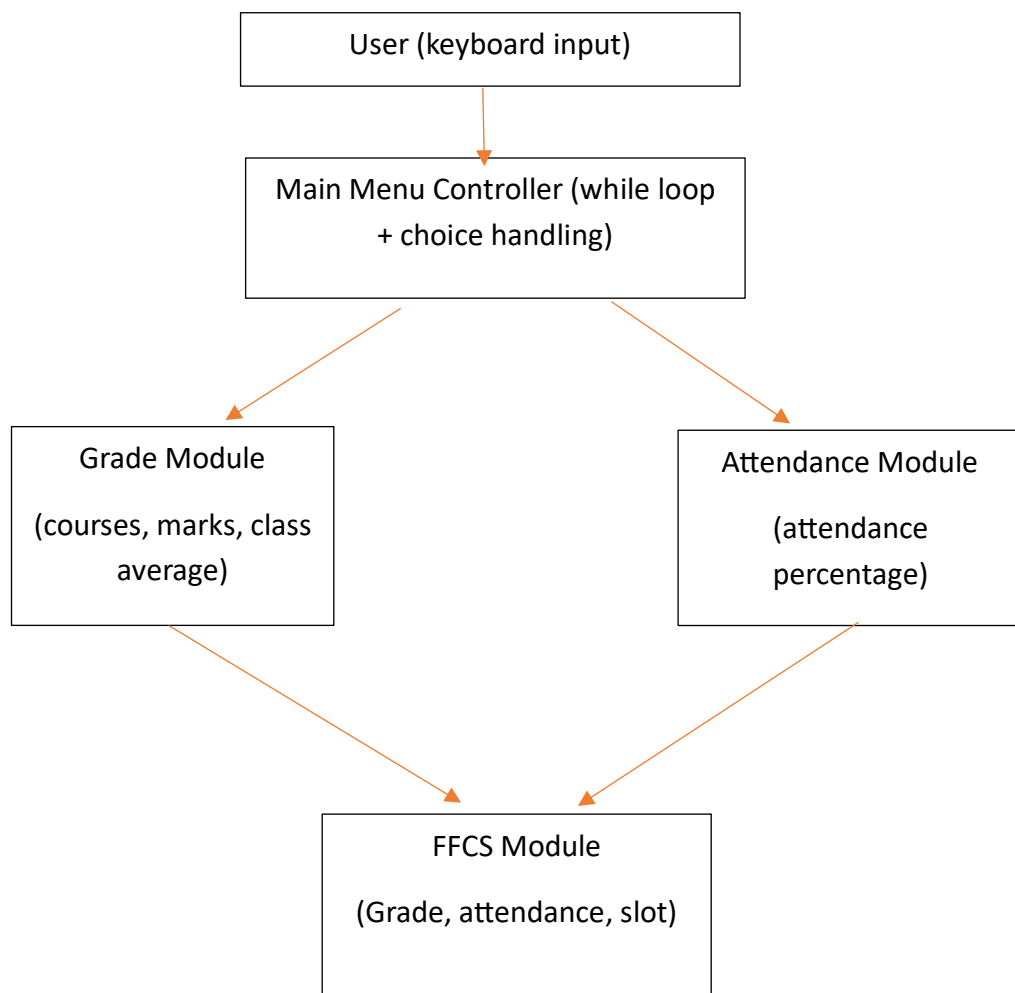
• Non-Functional Requirements

- **Usability**: the application uses clear console prompts and a numbered menu.
- **Simplicity**: the source is a single Python script and uses fundamental language features.
- **Portability**: no external libraries or database are referenced in the supplied source, so the program is designed around standard Python console I/O.
- **Maintainability**: the three major functions are separated logically through menu branches, although they are implemented in one script rather than separate functions/modules.
- **Performance**: the calculations are small and operate on user-entered values, so no substantial computational workload is present.
- **Data persistence**: none is implemented; course and attendance information exists only during the current execution.

• System Architecture

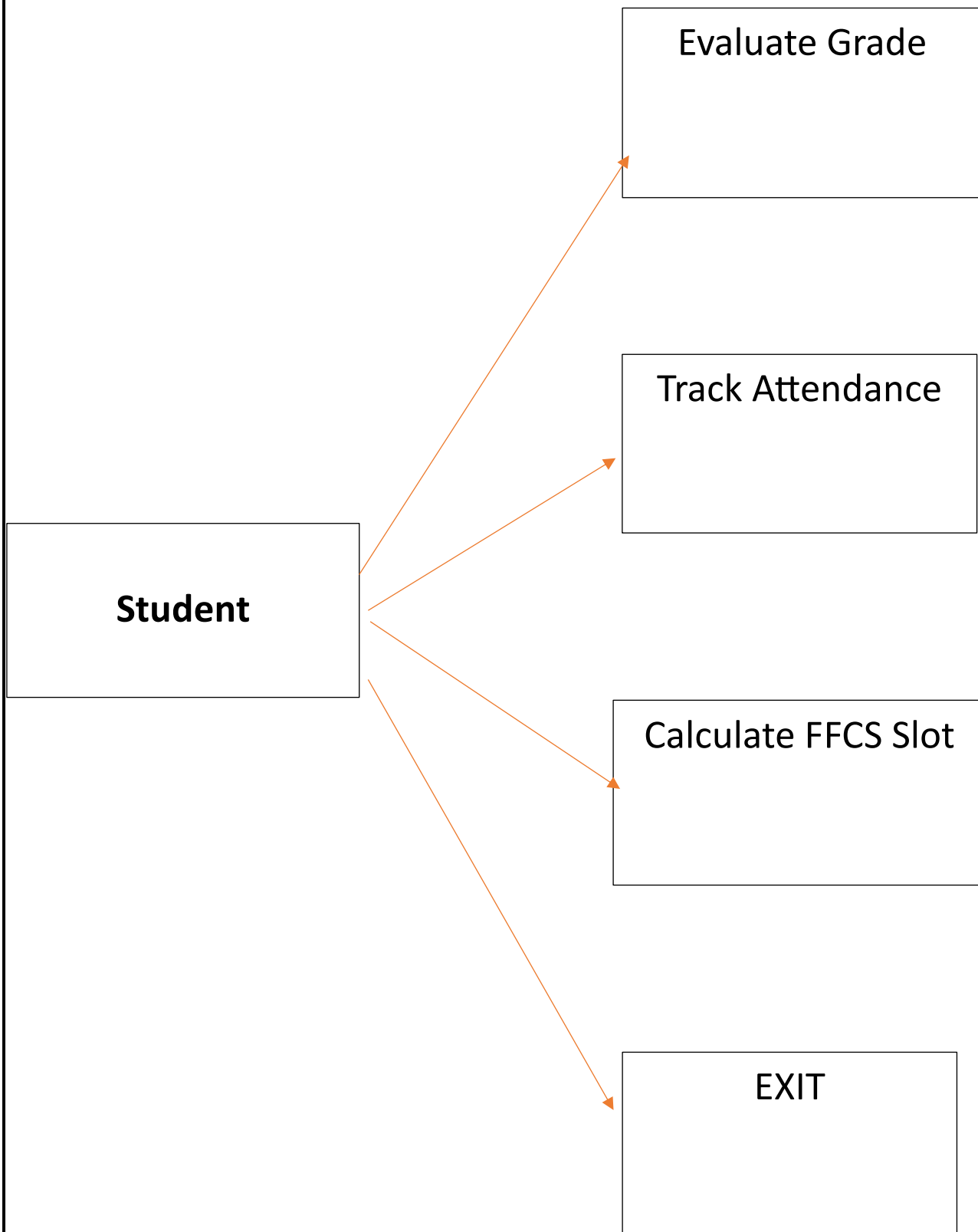
The supplied implementation follows a simple single-process, menu-driven architecture. There is no client/server layer, persistent storage layer, or external service. User input is read from the console, processed inside conditional branches, and the result is printed back to the console.

Architecture flow: Student → Main Menu Controller → selected utility → calculation/decision logic → console result → Continue/Exit.

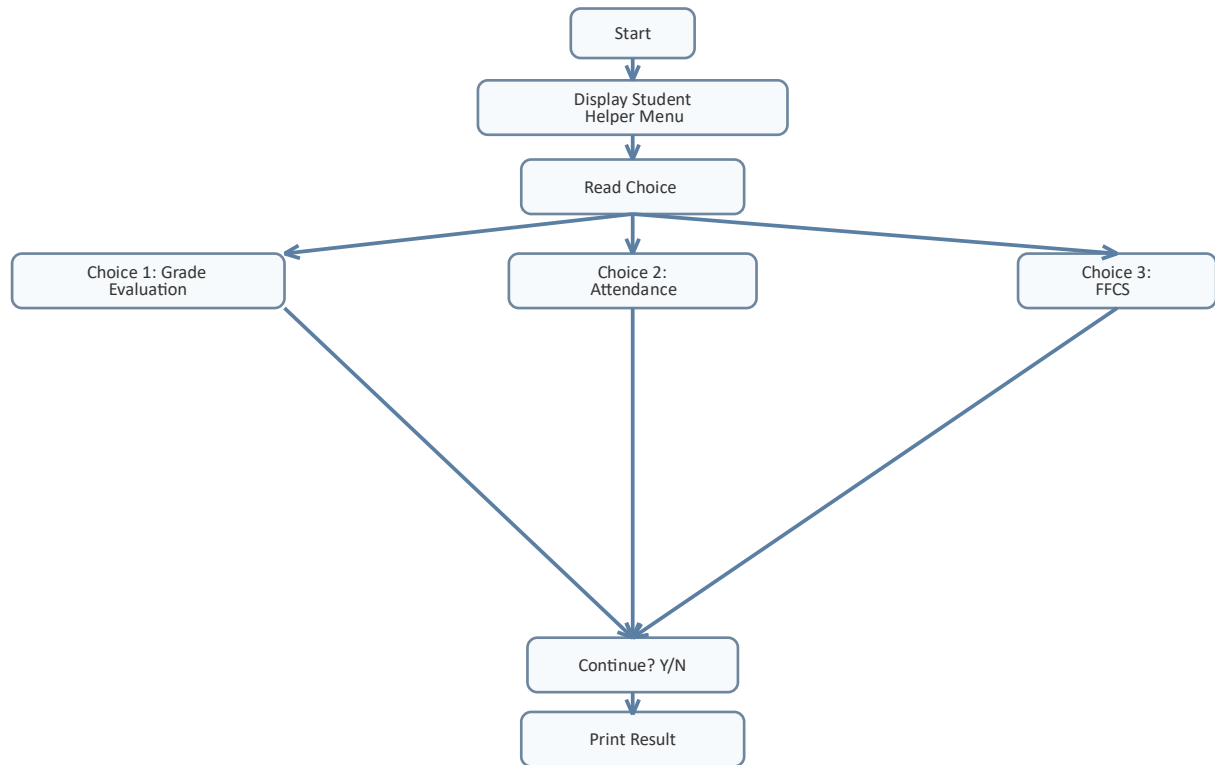


• Design diagrams

1. USE CASE DIAGRAM



2. WORKFLOW DIAGRAM



3. CLASS / COMPONENT DIAGRAM

<u>Component</u>	<u>Responsibility</u>
<u>Main loop</u>	<u>Displays menu, reads choice, repeats or exits.</u>
<u>Grade evaluation branch</u>	<u>Stores courses, marks, averages and calculated grades in lists.</u>
<u>Attendance branch</u>	<u>Calculates percentage and attendance requirements.</u>
<u>FFCS branch</u>	<u>Matches grade + attendance conditions to an FFCS slot.</u>
<u>Console I/O</u>	<u>Collects user input and displays results.</u>

• Design Decisions & Rationale

- Console interface: the source uses `input()` and `print()`, making the application straightforward to run from a Python terminal.
- Menu-driven design: one top-level choice routes the user to one of three student utilities.
- Lists for course data: `all_courses`, `mis`, `ais`, and `grade` store parallel per-course information during grade evaluation.
- **Conditional grading**: the grade is determined by comparing the student's mark with the class average, rather than by a fixed absolute mark threshold.
- **75% attendance target**: the attendance branch explicitly uses 75/100 in its semester-end calculation.
- **Direct FFCS mapping**: grade and attendance are handled with a chain of `if/elif` conditions, keeping the mapping visible in the source.

- **Implementation Details**

The report card then prints each course, scored mark, class average, and grade.

- **ATTENDANCE TRACKING**

Current attendance is calculated as **(classes attended / classes scheduled) × 100**. The program also calculates a semester-end 75% target using the current scheduled classes plus upcoming classes, then derives how many upcoming classes need to be attended and how many may be missed under the implemented arithmetic.

- **FFCS SLOT CALCULATION**

The FFCS branch reads total attendance and a grade, converts the grade to lowercase, and checks grade/attendance combinations. It prints slot numbers from 1 through 8 for the combinations explicitly represented in the source.

- **Program Control**

After each selected operation, the program asks **PRESS Y/N ENTER TO CONTINUE**. A Y/y input continues the outer loop. An N/n input causes the loop condition to fail and the program prints **THANK YOU**.

• Screenshots / Results

```
= RESTART: C:\Users\hpl\AppData\Local\Programs\Python\Python314\correctedmain_done.py
```

```
~~~~~  
STUDENT HELPER  
~~~~~
```

```
|1| GRADE EVALUATION.  
|2| TRACK YOUR ATTENDANCE.  
|3| FFCS  
~~~~~
```

```
ENTER YOUR PREFERENCE=>1  
ENTER NUMBER OF COURSES REGISTERED=>3  
enter course '1'=>MAT1003  
enter your mark in 'MAT1003' =>45  
enter your class avg. in 'MAT1003'=>34  
enter course '2'=>CSE1021  
enter your mark in 'CSE1021' =>48  
enter your class avg. in 'CSE1021'=>33  
enter course '3'=>CHY1006  
enter your mark in 'CHY1006' =>36  
enter your class avg. in 'CHY1006'=>21
```

```
~~~~~  
REPORT CARD  
~~~~~
```

```
course is MAT1003  
Scored mark=> 45  
Class avg.=> 34.0  
Grade=> S
```

```
course is CSE1021  
Scored mark=> 48  
Class avg.=> 33.0  
Grade=> S
```

```
~~~~~
ENTER YOUR PREFERENCE=>1
ENTER NUMBER OF COURSES REGISTERED=>3
enter course '1'=>MAT1003
enter your mark in 'MAT1003' =>45
enter your class avg. in 'MAT1003'=>34
enter course '2'=>CSE1021
enter your mark in 'CSE1021' =>48
enter your class avg. in 'CSE1021'=>33
enter course '3'=>CHY1006
enter your mark in 'CHY1006' =>36
enter your class avg. in 'CHY1006'=>21
```

```
~~~~~
REPORT CARD
~~~~~
```

```
course is MAT1003
Scored mark=> 45
Class avg.=> 34.0
Grade=> S
```

```
~~~~~
course is CSE1021
Scored mark=> 48
Class avg.=> 33.0
Grade=> S
```

```
~~~~~
course is CHY1006
Scored mark=> 36
Class avg.=> 21.0
Grade=> S
```

```
~~~~~
PRESS Y/N ENTER TO CONTINUE=>|
```

```
~~~~~
STUDENT HELPER
~~~~~
```

- ```
[1] GRADE EVALUATION.
[2] TRACK YOUR ATTENDANCE.
[3] FFCS
```

```
~~~~~
ENTER YOUR PREFERENCE=>2
ENTER TOTAL NUMBER OF CLASSES ATTENDED =>22
ENTER TOTAL NUMBER OF CLASSES SCHEDULED =>30
ENTER NUMBER OF UPCOMING CLASSES =>10
YOU ARE CURRENTLY BELOW THE CRITERIA WITH: 73.33333333333333 % ATTENDANCE
YOU HAVE TO ATTEND '8.0' CLASSES TO MAINTAIN 75% ATTENDANCE
PRESS Y/N ENTER TO CONTINUE=>y
```

```
~~~~~
STUDENT HELPER
~~~~~
```

- ```
[1] GRADE EVALUATION.
[2] TRACK YOUR ATTENDANCE.
[3] FFCS
```

```
~~~~~
ENTER YOUR PREFERENCE=>2
ENTER TOTAL NUMBER OF CLASSES ATTENDED =>28
ENTER TOTAL NUMBER OF CLASSES SCHEDULED =>30
ENTER NUMBER OF UPCOMING CLASSES =>10
YOU ARE CURRENTLY ABOVE THE CRITERIA WITH: 93.33333333333333 % ATTENDANCE
YOU HAVE TO ATTEND '2.0' CLASSES TO MAINTAIN 75% ATTENDANCE
bunk 8.0
PRESS Y/N ENTER TO CONTINUE=>y
```

```
~~~~~  
ENTER YOUR PREFERENCE=>2  
ENTER TOTAL NUMBER OF CLASSES ATTENDED =>22  
ENTER TOTAL NUMBER OF CLASSES SCHEDULED =>40  
ENTER NUMBER OF UPCOMING CLASSES =>0  
your attendance is=> 55.000000000000001 R.I.P  
PRESS Y/N ENTER TO CONTINUE=>y
```

```
~~~~~  
STUDENT HELPER
~~~~~
```

```
|1| GRADE EVALUATION.  
|2| TRACK YOUR ATTENDANCE.  
|3| FFCS  
~~~~~
```

```
ENTER YOUR PREFERENCE=>3
Enter Your Total attendance=>89
Enter your Grade=>s
FFCS SLOT => 2
PRESS Y/N ENTER TO CONTINUE=>y
```

```
~~~~~  
STUDENT HELPER  
~~~~~
```

```
|1| GRADE EVALUATION.
|2| TRACK YOUR ATTENDANCE.
|3| FFCS
~~~~~
```

```
ENTER YOUR PREFERENCE=>3  
Enter Your Total attendance=>98  
Enter your Grade=>a  
FFCS SLOT => 1  
PRESS Y/N ENTER TO CONTINUE=>n  
THANK YOU  
|
```

## • Testing Approach

The testing approach is organized around menu coverage, boundary values, and invalid/edge conditions. The expected outcomes below are derived from the current logic.

Important boundary observations from the source: the attendance branch checks a  $< 75\%$  and a  $> 75\%$  condition, so exactly 75% is not covered by either branch and falls to the final else. The FFCS branch also contains a repeated D-grade condition near the end and does not include a separate F-grade mapping. These are source-level behaviours to consider during testing



|                                                               |  | Expected behavior                                                                     |
|---------------------------------------------------------------|--|---------------------------------------------------------------------------------------|
| Input / Scenario                                              |  |                                                                                       |
| Choose 1; enter one course and mark at least 10 above average |  | Grade branch should assign S.                                                         |
| Choose 1; mark exactly average + 5                            |  | Grade branch should assign A.                                                         |
| Choose 1; mark within the implemented B interval around the a |  | verageGrade branch should assign B.                                                   |
| Choose 2; attended < 75%                                      |  |                                                                                       |
| Choose 2; attended > 75%                                      |  | Program enters the above-criteria message when its additional condition is satisfied. |

|                                            |                                    |
|--------------------------------------------|------------------------------------|
| Choose 3; grade S and attendance $\geq 90$ | Prints FFCS SLOT => 1.             |
| Choose 3; grade A and attendance 75–84.99  | Prints FFCS SLOT => 3.             |
| Choose 3; grade D and attendance 80–84.99  | Prints FFCS SLOT => 6.             |
| Choose invalid menu option                 | Prints “Enter Valid Choice!”.      |
| After an operation enter N/n               | Exits loop and prints “THANK YOU”. |

## • Challenges Faced

- **Multiple input types**: the program accepts integers for menu choices and counts, floats for class averages and attendance, and strings for course names and grades.
- **Boundary conditions**: grade and attendance decisions depend on ranges, so exact threshold values require careful testing.
- **Parallel lists**: grade evaluation keeps related course information in separate lists and relies on matching indexes.
- **Complex FFCS branching**: the long if/elif chain makes it important to keep grade and attendance intervals consistent.
- **Input validation**: the current implementation assumes numeric input can be converted successfully; invalid text for integer/float prompts can raise a Value Error.
- **Documentation consistency**: some inline comments describe conditions inaccurately compared with the actual code, so the executable statements are the authoritative implementation.

## • Learnings and Key Takeaways

- Applied Python fundamentals: variables, lists, loops, conditionals, input/output, arithmetic, and string methods.
- Learned how nested/ordered if-elif conditions implement rule-based decision systems.
- Practiced collecting repeated records with a for loop and storing them in lists.
- Understood how attendance percentages and target calculations can be translated into program logic.
- Observed the importance of boundary-value testing for grading and attendance rules.
- Gained experience designing a small menu-driven command-line application.

## • Future Enhancement

- **Refactor into functions**: create separate functions such as `grade_evaluation()`, `attendance_tracker()`, and `ffcs_calculator()`.
- **Improve validation**: safely handle non-numeric input, invalid grades, negative values, and attendance above 100%.
- **Fix boundary behavior**: explicitly handle exactly 75% attendance and review all grade/attendance endpoints.
- **Use structured data**: replace parallel lists with dictionaries or a small `Course` class/data structure.
- **Add persistence**: save student/course information in a file or database if required.
- **Add a GUI**: provide forms and buttons for users who prefer a graphical interface.
- **Improve FFCS rules**: represent slot rules as structured data so they are easier to update and test.
- **Add automated tests**: use Python's unit test or `python test` to test each grade, attendance, and FFCS boundary.
- **Improve reporting**: export the report card and attendance summary to CSV/PDF

## • References

- **Supplied project source:** *#main.py*, provided with the project materials. The report's functional and implementation descriptions are based on this source.
- **Supplied submission requirements:** reference images showing the required project-report sections, including Cover Page, Introduction, Problem Statement, Functional Requirements, Non-functional Requirements, System Architecture, Design Diagrams, Design Decisions & Rationale, Implementation Details, Screenshots / Results, Testing Approach, Challenges Faced, Learnings & Key Takeaways, Future Enhancements, and References
- **Python language concepts used in the supplied source:** console input/output, lists, for/while loops, conditional statements, numeric conversion, formatted strings, and string lowercase conversion. No external library reference appears in the supplied source.